NFE106 – Administration de bases de données
Examen 3 février 2020 – session 1 – 2h30 (3 pages)

Consignes : cet examen est exceptionnellement organisé à distance. Vous devez rendre votre copie numérisée à 11h45 au plus tard. Aucune copie ne sera acceptée après cette heure limite. Les seuls formats acceptés sont le PDF et le JPEG, ceci afin d'éviter tout problème de lecture. Prenez vos dispositions pour être en mesure de fournir le bon format le moment venu. Une activité de discussion en direct (« *chat* ») est ouverte dans l'espace Moodle pour d'éventuelles questions.

1 Stockage et indexation (8 pts)

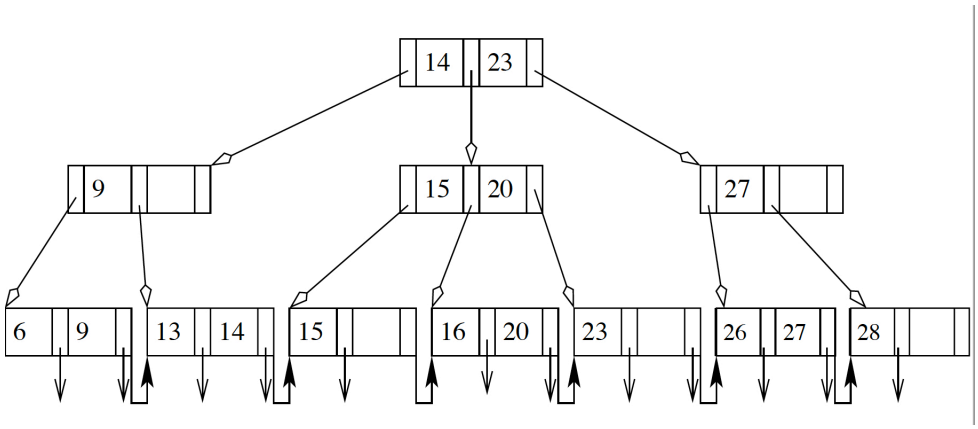

FIGURE 1 – Un arbre B

1. Une table relationnelle contient 256 500 lignes. On la stocke dans une base avec des blocs de 4 000 octets. On suppose que chaque enregistrement a une taille de 200 octets et que chaque bloc a un entête de 100 octets. Quel est le nombre d'enregistrements par bloc ? Quel est le nombre de blocs nécessaire pour stocker la table (on suppose que les blocs contiennent autant d'enregistrements que possible) ?
2. Expliquez la différence entre un index dense et un index non-dense. Combien d'index denses peut-on construire sur un fichier ? Combien d'index non-denses ?
3. La figure 1 montre un arbre B. On peut stocker au plus 2 entrées par bloc. Indiquez le rôle des différents pointeurs représentés, et donnez l'arbre après insertion d'un enregistrement de clé 17 (dessinez au moins les parties de l'arbre qui ont changé).
4. Un arbre B indexe un fichier de 300 enregistrements. On suppose que chaque nœud stocke au plus 10 clés et 10 pointeurs. Quelle est la hauteur minimale de l'arbre et sa hauteur maximale ? (Un arbre constitué uniquement de la racine a pour hauteur 0).
5. Inversement, on constate que l'arbre B a pour hauteur 1. Quel est le nombre minimal de pointeurs par bloc ? Le nombre maximal ?
6. On indexe une table par un arbre B sur un identifiant dont les valeurs sont fournies par une *séquence*. À chaque insertion un compteur est incrémenté et fournit la valeur de clé de l'enregistrement inséré. On suppose qu'il n'y a que des insertions dans la table. Montrez que tous les nœuds de l'index qui ont un frère droit sont exactement à moitié pleins.

2 Optimisation (8 points)

Soit les tables relationnelles suivantes (les attributs qui forment une clé sont en gras) :
— Produit (**code**, nom, marque, prix)
— Client (**id**, nom, prénom)
— Achat (**codeProduit**, **idClient**, date, quantité)
On donne ci-dessous une requête SQL et le plan d'exécution fourni par Oracle :

```
select quantité
from Produit as p, Client as c, Achat as a
where p.code = a.codeProduit
and   c.id = a.idClient
and   prix > 50
```

Plan d'exécution :

```
0 SELECT STATEMENT
  1 MERGE JOIN
    2 SORT JOIN
      3 NESTED LOOPS
        4 TABLE ACCESS FULL ACHAT
        5 TABLE ACCESS BY INDEX ROWID PRODUIT
          6 INDEX UNIQUE SCAN A34561
    7 SORT JOIN
      8 TABLE ACCESS FULL Client
```

1. Que peut-on déduire de ce plan : peut-on savoir s'il existe un index sur la table Client ? Sur la table Produit ? Sur la table Achat ? Si vous pensez qu'un index existe, donnez les attributs indexés. Justifiez vos réponses.
2. Quels sont les algorithmes de jointure utilisés par Oracle ? Expliquer le fonctionnement de ce plan d'exécution.
3. Quel(s) index peut-on ajouter pour obtenir les meilleures performances possibles ? Donnez, sous la forme que vous souhaitez, le plan d'exécution après ajout d'index.
4. On fait une jointure entre les tables Produit et Achat. Produit occupe 500 blocs, Achat 10 000 blocs. Vaut-il mieux appliquer un algorithme de boucles imbriquées ou de hachage si la mémoire M disponibe est de 600 blocs ?

3 Questions de cours (4 points)

Parmi les requêtes suivantes, quelles sont celles qui nécessitent un opérateur bloquant. (Aide : se demander s'il faut lire ou non toute la table avant de produire le résultat).

1. select titre from Film
2. select distinct titre from Film
3. select count(titre) from Film group by année
4. select titre from Film order by année
5. select substring(titre, 1, 10) from Film

Pourquoi la clé primaire d'une table doit-elle être indexée (plusieurs réponses possibles) :

1. Parce que la plupart des requêtes SQL portent sur la valeur de la clé primaire.

2. Pour vérifier rapidement la contrainte d'unicité lors d'une insertion.
3. Pour vérifier rapidement la contrainte d'intégrité référentielle lors de l'insertion d'un clé étrangère.
4. Pour vérifier rapidement la contrainte d'intégrité référentielle lors de la destruction d'une clé primaire.